

Milestone 2 - Learned Compatibility for Jigsaw Reconstruction

CSE480 Machine Vision . Ain Shams University, Faculty of Engineering, Mechatronics Engineering Department . Summer 2026

1. Overview

Milestone 1 reconstructed a jigsaw from a hand-designed compatibility measure: shape continuity plus colour continuity, combined with fixed weights. This milestone replaces that measure with two **learned** ones - a Siamese convolutional network and a graph neural network - and compares all three on the same puzzles.

The comparison is deliberately narrow, and that is the point. Segmentation, contour tracing, corner location, side description and **the assembly algorithm itself** are all unchanged from Milestone 1. The only thing that differs between the three methods is the function that answers "*how well do these two sides fit?*". Everything downstream consumes the same CompatibilityTable interface, so any difference in the reconstructions is attributable to the matcher and not to a different pipeline.

```
segment -> describe pieces -> ??? classical: shape + colour formula ???  
                                     ??? Siamese CNN                               ??-> same assembler -> image + quality  
                                     ??? graph neural network                       ??
```

Both models are trained on the provided data, as Task 1 requires: the 50 photographs of the actual jigsaw under detection/. That is not free - the export labels each piece's identity, not which of its sides touches which - so §2.1 sets out how the missing side-level labels are recovered from the annotations and verified. An earlier version of this milestone trained on generated puzzles instead, having concluded the recovery was impossible; correcting it roughly doubles both models' accuracy on the real photographs (§6.4), and is the single most consequential change in this report.

2. Task 1 - Dataset preparation

2.1 Recovering side-level labels from the provided data

A training example is a **pair of sides** labelled 1 if they are genuinely adjacent in the finished puzzle and 0 otherwise. Producing that label requires knowing, for every side of every piece, which grid direction it faced before the pieces were shuffled.

The provided annotations do not state it. They give each piece's *identity*, which - as Milestone 1 §9.1 established - fixes which **pieces** are neighbours, but not which of a piece's four sides touches which, because the rotation a piece happens to be lying at on the cloth is unrecorded.

An earlier version of this report concluded that side-level labels were therefore "simply not recoverable from a bounding box and a class id", and trained on generated puzzles instead. That was wrong, and it was the most consequential error in this milestone - §6.4 shows the models roughly double their accuracy on the real photographs once it is corrected. What is missing is one integer per piece, and two independent constraints determine it.

src/ml/real_labels.py recovers it. A piece's rotation is written, as everywhere in this project, as the index of the side facing North; side *s* then faces direction $(s - \text{rotation}) \bmod 4$.

1. **The border.** The identity gives the cell, so a piece on the rim of the grid *must* present a flat side outwards and an interior side must not be flat. This pins every border and corner piece outright - 20 of the 35. Crucially the rotation is chosen by each side's continuous amplitude rather than by its tab/blank/flat **label**, because on these photographs that label is right only about 80 % of the time (Milestone 1 §9.3) and only about half the true corner pieces are recognised as corners. Using the measurement instead of the classification makes the recovery independent of the classifier's failure.

2. **Complementarity.** Interior pieces carry no flats, so constraint 1 says nothing about them. They are pinned by propagating outwards from pieces already fixed, breadth-first, choosing the rotation whose seam against a fixed neighbour is tab-against-blank and cheapest under the classical measure. This is the same "most constrained first" ordering the assembler uses.

The recovery is checked, not trusted. Two independent measures, over all 50 photographs:

Check	Result	Chance
True seams that come out tab-against-blank	0.818	~0.25
Border sides that measure flattest	0.787	~0.25

Neither is forced by the procedure, so both are evidence that the recovered rotations are the real ones. The residual ~18 % tracks the flat/tab/blank classifier's own error rate, not a failure of the recovery.

A third check is stronger still, and is a unit test rather than a table. A *generated* puzzle carries both the cells and the true seams, so the recovery can be handed only the cells and asked to rediscover the seams the generator recorded: **over 80 % of the seams it returns are exactly right** (test_rotation_recovery_reproduces_the_known_seams).

Only seams that come out complementary are kept as positives, so the training set holds the ones the recovery is confident about - about 46 per photograph. A photograph whose complementary fraction falls below 0.6 is dropped entirely; 49 of 50 survive.

Every sample is passed through **the same pipeline the classical matcher uses** - segmented by thresholding, traced, corner-located and described - so the networks see the same segmentation noise, corner-localisation error and colour sampling as the measure they are compared against.

2.2 Positive and negative sampling

Positives are all the true seams of a puzzle. Negatives are drawn **only from admissible pairs** - a tab facing a blank on a different piece. This matters more than it looks: a model trained against arbitrary negatives would learn mostly to reject flat-to-flat and tab-to-tab pairings, score beautifully, and be worthless, because the classical matcher already excludes those for free with a hard constraint. The learned model must be better than chance *on the pairs that are genuinely in contention*.

Six negatives are drawn per positive, **half of them from the hardest available candidates** - the ones the classical measure ranks best - so the networks spend their capacity where the problem actually is rather than on easy rejections. Negatives are redrawn every epoch, so the models see far more of the negative space than a single fixed draw would give.

2.3 Augmentation

Applied to the colour channels only; the shape channel is geometry and recolouring it would be meaningless. The four effects match those the brief lists:

Effect	Range	Why
Illumination	$\times 0.85$ -1.15	pieces lie under different light across a table
Contrast	$\times 0.8$ -1.25 about the mean	exposure varies between photographs
Colour balance	$\times 0.93$ -1.07 per channel	white balance drifts
Sensor noise	$\sigma = 0.02$ additive	camera noise

2.4 Splitting

Splits are by **puzzle, never by pair**. Two sides of one seam must not land on opposite sides of the split, and all pieces of one puzzle share a source picture, so a pair-level split would leak. 70 / 15 / 15 train / validation / test. The identical `SplitSpec` object is handed to both models, and the test split is not touched until the final evaluation.

The set actually used. The **49 provided photographs** that could be labelled (of 50 that show a full scramble), split 34 / 7 / 8 by photograph. That yields **10,724 training pairs** (1,532 positive, 9,192 negative - a positive fraction of 0.143), with 2,289 for validation and 2,611 for test. Full record in `results/milestone2/dataset.json`.

Every photograph shows the same 5x7 jigsaw, so this set has no variation in puzzle size. The size study the brief asks for (§6.2) therefore comes from a second run on generated puzzles of three sizes, kept in `results/milestone2/`; `python main_milestone2.py` produces it and `python main_milestone2.py --real` the set described here.

3. Task 2 - Model development

The brief requires two *fundamentally different* models, and notes that changing layer counts, activations or hyper-parameters does not qualify. These two differ in their unit of inference, their input representation and their inductive bias.

3.1 Model 1 - Siamese convolutional network

Why this shape of model. A side's descriptor genuinely is a small picture: three colour channels sampled at eight depths just inside the cut and sixty-four positions along it, with the shape profile in a fourth channel and in register with them. Asking whether two such strips continue into one another is a texture and edge-continuation question, which is what convolutions are for. And the kernels must be the *same* for both sides, since "does A continue into B" cannot depend on which side was named A - that is exactly a Siamese network.

Input	two (4, 64, 8) tensors: RGB at 8 depths + shape profile. The second is reversed along its length, because mating sides are traversed in opposite directions
Encoder	three convolutional blocks (3×3 conv, BatchNorm, ReLU, twice, then max-pool along the side only), widening 4 -> 32 -> 64 -> 128; global average pool; linear to a 128-d embedding. Weights shared between the two inputs
Head	embeddings combined as <code>[</code>
Output	<code>sigma(logit)</code> = probability the two sides are neighbours
Parameters	345,473 (1.382 MB)

Pooling is applied along the length of the side only. The depth axis is just eight samples deep and each one means something different (distance from the cut), so pooling it away early would destroy the very gradient information that tells the model whether the picture is continuing.

3.2 Model 2 - Graph neural network

Why a different model, not a re-tuned one. The Siamese network judges a pair *in isolation*: two strips in, one probability out, with no knowledge of the rest of the puzzle. That is precisely its weakness, because a jigsaw is a competition - a side does not need a partner that looks plausible, it needs the partner that looks *more plausible than every other candidate*, and whose own best choice is this side in return.

The graph model is built on that observation:

- **the unit of inference is the whole puzzle**, not the pair. Every side of every piece is a node; every admissible pairing is a candidate edge; one forward pass scores all of them together.
- **the representation is relational, not convolutional**. Nodes carry a flat 45-number descriptor (pooled colour, the shape profile, the side type one-hot, the length and amplitude). There is no convolution anywhere in the model. What refines a node is **message passing** - each side sees its rival candidates and updates its own embedding accordingly, so the score it gives a partner is a contextual judgement rather than an absolute one.
- **sides of the same piece are linked**, so a side also knows what the rest of its own piece looks like - information the Siamese model never receives.

Input	one graph per puzzle: 4N nodes with 45-d features; edges are admissible inter-piece candidates (capped at 24 per side, shortlisted by the classical cost) plus the intra-piece links
Layers	3 × GraphSAGE convolution (mean aggregation), hidden width 96, LayerNorm + ReLU between
Edge head	a candidate (a, b) scored from $[h_a + h_b,$
Output	$\sigma(\text{logit})$ per candidate edge
Parameters	97,441 (0.39 MB) - 3.5x fewer than the Siamese

3.3 The four outputs the brief requires

For any candidate pair, each model reports: **whether they are neighbours** ($\sigma(\text{logit}) \geq 0.5$), a **numerical compatibility score** (that probability), **which sides matched** (the pair of side indices), and the **relative orientation** the pairing implies.

The last is derived, not predicted, and deliberately so - it is determined exactly by which sides matched. With each piece's rotation written as the index of the side facing North, side a facing direction d means $a = \text{rot_a} + d$, and its partner must face the opposite direction, $b = \text{rot_b} + d + 2$. Eliminating d:

$$\text{rot_b} - \text{rot_a} = b - a + 2 \pmod{4}$$

Predicting this with a network instead would add a way to be wrong about something that is already certain. A test asserts the identity holds for all 64 combinations of the two rotations and the direction.

4. Task 3 - Training

Both models are trained under an identical protocol; only the model and the representation it needs differ.

	Siamese CNN	Graph NN
Loss	BCE-with-logits, positive-weighted by the class ratio	same
Optimiser	Adam	Adam
Learning rate	1e-3, halved after 3 epochs without validation improvement	same
Weight decay	1e-4	1e-4
Batch	256 pairs	one puzzle graph (the natural unit)
Epochs	30, early stopping patience 8	same
Augmentation	illumination, contrast, colour, noise	same features, same augmentation
Model selection	best validation AUC	same

Why AUC and not accuracy. The classes are deliberately unbalanced 1:6, so accuracy is dominated by the negatives; and what the assembler actually consumes is a *ranking* of candidate partners rather than a thresholded decision. AUC measures exactly the ranking quality, and is what model selection and early stopping key on. It is computed from rank statistics (the Mann-Whitney form) and is unit-tested against scikit-learn.

Positive weighting is set to the actual negative/positive ratio each epoch, so the loss does not simply learn to answer "not neighbours".

4.1 Training results

	Siamese CNN	Graph NN
Best validation AUC	0.9016 (epoch 27)	0.8658 (epoch 27)
Final training AUC	0.9516	0.8823
Final validation AUC	0.9003	0.8615

	Siamese CNN	Graph NN
Final training loss	0.4695	0.8356
Final validation loss	0.3683	0.4249
Epochs run	30	30
Wall-clock training	1495 s	30 s
Diagnosis	healthy (val AUC 0.902, train-val gap 0.047)	healthy (val AUC 0.866, train-val gap 0.018)

Real seams are far harder than generated ones, and the curves say so. On generated puzzles validation AUC passed 0.89 within a single epoch; here the Siamese starts at 0.61 and needs 27 epochs to reach 0.902. That gap is the honest difficulty of the actual dataset - a nearly white picture, per-piece illumination, and labels recovered rather than known by construction.

Over- or under-fitting. Neither, though the Siamese runs closer to the line than it did on generated data: its train/validation AUC gap at the selected epoch is 0.042 against the graph model's 0.017, and its training AUC continues to climb after validation AUC flattens. That is mild over-fitting beginning, and it is exactly what selecting the best-validation checkpoint rather than the last one is for. With 34 training photographs of one jigsaw the risk is real and worth naming rather than glossing. Note that validation AUC sits *above* training AUC for the early epochs, which looks wrong until one remembers that the training batches are augmented and half their negatives are mined from the hardest available candidates, while validation is neither - the training set is deliberately the harder of the two.

The training-cost gap is the first real difference between the models. The Siamese needs 50x longer to train (1495 s versus 30 s) for 3.5x the parameters, because it convolves over a 2,048-value tensor per side while the graph model consumes a 45-number descriptor and scores a whole puzzle in one pass.

5. Task 4 - Puzzle assembly

Each model's scores are converted to costs and handed to the **same** assemble routine used in Milestone 1 - with its border constraints, its best-buddy-first tie-breaking, its three-stage dead-end relaxation and its guarantee to return the best arrangement obtained. Each piece is used exactly once and border constraints are respected because those are properties of the assembler, not of the matcher.

The conversion from a probability to a cost is

$$\text{cost} = -\log(p + \text{epsilon})$$

which is the negative log-likelihood of the pairing. It is chosen over $1 - p$ for three reasons: it is monotonically decreasing in p , so the ordering the model intended is preserved exactly; it is **additive** across the seams of an arrangement, so the total the assembler minimises is the negative log-likelihood of the whole arrangement, which is the quantity that ought to be minimised; and it diverges as $p \rightarrow 0$, which keeps hopeless pairings out of the search entirely.

Admissibility - a flat never forms an interior seam, a tab must meet a blank - is applied on top of the model's opinion exactly as for the classical table, so all three methods search the same space.

6. Task 5 - Performance evaluation

All three methods are evaluated on the **same unseen test puzzles**, which neither model saw during training or model selection.

6.1 The standard test split

Measure	Classical	Siamese CNN	Graph NN
Matching top-1	0.111	0.438	0.263
Matching AUC	0.718	0.944	0.880

Measure	Classical	Siamese CNN	Graph NN
Best-buddy precision	0.222	0.476	0.300
Neighbour accuracy	0.165	0.295	0.239
Position accuracy	0.131	0.173	0.147
Orientation accuracy	0.371	0.388	0.370
Complete reconstructions	0/8	0/8	0/8
Parameters	0	345,473	97,441
Model size	0 MB	1.382 MB	0.39 MB
Training time	0 s	1495 s	30 s
Inference / puzzle (s)	0.042	0.226	0.038

This table is not saturated, and that is the point of using the provided data. On generated puzzles every method reconstructed 8 of 9 test puzzles and the comparison could not separate them (§6.2 keeps that run for the size study). Here **no method completes a single one of the 8 held-out photographs** - and precisely because nothing is at the ceiling, the ordering is unambiguous. The Siamese ranks candidates four times better than the classical formula (top-1 0.438 against 0.111, AUC 0.944 against 0.718) and leads on every reconstruction measure.

Orientation accuracy is the weakest column for every method (0.37-0.39), and it should be. A piece counts as correctly oriented only when the side facing North matches the one that faced North in the finished puzzle, under a single global quarter turn shared by every piece, since an arrangement that comes out sideways is still an arrangement. On a puzzle the matchers are largely failing to place, orientation sits near what chance over four rotations would give. Reporting it separately from position is what makes that visible: a piece can occupy the correct cell turned the wrong way, and the position measure alone calls that a success, which is presumably why the brief asks for both.

The ground truth for it is not stored anywhere. It is implied by the labels already present: a piece's rotation is the index of the side facing North, and every true seam pins that down, because the grid says which way the seam runs. `PuzzleSample.true_rotations` derives it, and a test checks the derivation against the generator's own record for every piece of a puzzle.

6.2 How performance changes with puzzle size

Every provided photograph shows the same 5x7 jigsaw, so the training set of §2.4 cannot vary puzzle size at all. On it, the one available row is

Grid	Classical	Siamese	Graph NN
5x7 (provided photographs)	0.16	0.30	0.24

The brief nonetheless asks how performance changes with size, so that question is answered by a second run on generated puzzles of three sizes, recorded in `results/milestone2/scaling_generated.json` (python `main_milestone2.py`, without `--real`). Neighbour accuracy, and matching AUC underneath it:

Grid	Classical	Siamese	Graph NN	t class.	t Siam.	t GNN
3x4	0.926 / 1.00	0.980 / 1.00	0.970 / 1.00	0.008	0.060	0.010
4x5	0.971 / 1.00	0.994 / 1.00	0.976 / 1.00	0.019	0.103	0.019
5x7	0.932 / 0.91	0.987 / 0.97	0.957 / 0.86	0.039	0.180	0.025

Accuracy falls with size, and that is where the methods separate. At 3x4 and 4x5 every method reconstructs every generated puzzle perfectly and the comparison is saturated. At 5x7 - 35 pieces, the size of the real jigsaw - the ceiling breaks and an ordering appears: Siamese 0.97, classical 0.91, graph 0.86. That ordering is the same one the provided photographs give in §6.1, on data an order of magnitude harder, which is the useful cross-check.

Cost grows quadratically. Matching time roughly quadruples from 3x4 to 5x7 for every method, because the number of candidate side pairs goes as the square of the piece count. The graph model is cheapest at every size;

the Siamese is five to seven times dearer than either.

6.3 The texture sweep, and why it no longer measures what it did

The sweep fades a *generated* picture towards flat grey and asks each method to reconstruct it. When the models were trained on generated puzzles this was the one controlled test that could break the saturated ceiling. Now that they are trained on the provided photographs it measures something else entirely, and the numbers have to be read accordingly:

Texture	Classical	Siamese CNN	Graph NN
1.00	1.00	0.54	0.48
0.60	1.00	0.42	0.48
0.35	1.00	0.23	0.34
0.20	1.00	0.31	0.27
0.10	0.84	0.25	0.30

This is a domain shift, not a robustness result. The classical formula is unchanged and needs no training, so it behaves exactly as in Milestone 1. The two networks, however, have never seen a generated puzzle: they were fit to photographs of one physical jigsaw and are here being asked about synthetic pictures with different colours, a different cut and no camera noise. Their low scores measure the distance between those two domains and say nothing about how they behave on the data they were trained for, which §6.1 and §6.4 report directly.

It is kept because the previous version of this report drew a headline conclusion from it and that conclusion deserves its correction on the record. That version ran **four** puzzles per level, found the Siamese perfect at texture 0.10 where the classical formula fell to 0.84, and concluded that learning buys robustness to a vanishing picture. Re-running the identical sweep at **twenty** puzzles per level reversed it - with one reconstruction worth 25 % of the old score, the earlier table was reporting sampling noise as a finding. The lesson survives the change of training set: an experiment intended to break a ceiling has to be powered well enough to support the conclusion drawn from it.

6.4 The real dataset photographs, and what training on them changed

This is the headline result of the milestone, and it is a reversal.

All three methods are run on eight full-scramble photographs, scored against the Milestone 1 §9.1 answer key. The classical baseline uses the full `main.DATASET_SOLVER` settings and every method shares the same `border_mode="soft"` assembly, so only the matcher differs. The two learned columns are shown twice: once for models trained on **generated** puzzles, and once for the same architectures trained on the **provided** photographs (§2.1).

Method	Neighbour (generated)	Neighbour (provided)	Position (generated)	Position (provided)
Classical	0.208	0.208	0.118	0.118
Siamese CNN	0.218	0.374	0.070	0.296
Graph NN	0.150	0.245	0.107	0.134

Training on the data the brief specifies roughly doubles both learned methods. The Siamese goes from 0.218 to **0.374** neighbour accuracy and from 0.070 to **0.296** position accuracy - the latter more than four times better, and now more than twice the classical measure's 0.118. The graph model improves from 0.150 to 0.245. The classical column is identical by construction: it has nothing to train.

The previous conclusion was wrong, and its cause is identifiable. That version of this section read: *"None of the three reconstructs the real puzzle, and none of them is meaningfully ahead... Learning does not help here."* The first half is still true - 0.374 is not a solved puzzle. The second half was an artefact of training on the wrong data. The same report also said "the binding constraint is the training distribution rather than the model", which was exactly right; what it did not do was act on it, because §2.1 had wrongly concluded the provided photographs could not be labelled at all.

So the honest statement is now narrower and stronger than either version. **Learning the compatibility measure helps substantially on this puzzle, provided it is learned from this puzzle** - and it still does not solve it, because a seam whose two sides are both white carries little information no matter what reads it. Milestone 1 measured that directly: 90 % of the printed area is below 0.10 chroma.

One caution on the comparison. Every photograph shows the *same* physical jigsaw, so a model trained on 34 of them and tested on 8 others has seen different scatterings, lightings and segmentations of the very same 35 pieces. The split is honest - it is by photograph, and no side of a test photograph is seen in training - but it measures generalisation across *photographs of one puzzle*, not across puzzles. A model trained this way would not transfer to a jigsaw it had never seen, whereas the classical measure would. That is a real limitation of what the provided data can support, and it is the price of the accuracy gain above.

The reconstruction-quality column of `comparison.json` must still not be read across methods: it normalises by each method's own cost table, and the learned tables are $-\log p$ while the classical one is a weighted RMS distance, so the three are in different units.

6.5 Which method is best

The brief asks for the most accurate and the most computationally efficient method. They are different methods, and both answers are now unambiguous because nothing is at the ceiling.

Most accurate: the Siamese CNN, clearly. On the unseen test split it leads on every measure - matching top-1 0.438 against the classical 0.111 and the graph model's 0.263, AUC 0.944 against 0.718 and 0.880, neighbour accuracy 0.295 against 0.165 and 0.239. On the eight scored photographs it reaches 0.374 neighbour and 0.296 position accuracy, against the classical measure's 0.208 and 0.118. It costs 345 k parameters, 1.4 MB and 25 minutes of training, and the accuracy holds only for this jigsaw (§6.4).

Most efficient: the graph neural network, and by a wide margin. It reaches AUC 0.880 - well above the classical 0.718, if below the Siamese - using **3.5x fewer parameters** (97 k, 0.39 MB), **50x less training time** (30 s against 1495 s) and inference *as fast as the hand-written classical formula* (0.038 s against 0.042 s per puzzle, where the Siamese needs 0.226 s). For a matcher that must run inside a search loop it is the practical choice.

Still worth keeping: the classical measure. It needs no training data, no training time and no parameters; it is the only one of the three that would work on a jigsaw it had never seen; and it remains the most robust as a picture fades, since it has no training distribution to be shifted away from. On the provided photographs it is now clearly last on accuracy, and that is the correct conclusion to draw once the learned models are trained on the right data.

6.6 What was learned

1. **Train on the data the task is about.** This is the whole milestone. The

same two architectures, unchanged, roughly double their accuracy on the real photographs when trained on those photographs instead of on generated puzzles (§6.4). Every other variable - architecture, loss, optimiser, augmentation, the assembler - was held constant.

2. **"The labels are not available" deserved more scrutiny than it got.** The

provided annotations really do not state which side touches which; the earlier version of this report stopped there. But the missing quantity was one integer per piece, and two constraints already present in the data - where a known cell's flats must lie, and that a tab must meet a blank - determine it. Checking a stated impossibility cost far less than the accuracy it was silently costing.

3. **Verify a recovered label before building on it.** The rotation recovery

is checked three ways: true seams come out complementary 0.818 of the time against 0.25 by chance, border sides measure flattest 0.787 of the time, and on a generated puzzle - where the answer *is* known - over 80 % of the seams it returns are exactly right. Only the seams that pass are used.

4. **A saturated benchmark hides everything.** On generated puzzles all three

methods reconstructed 8 of 9 test puzzles and could not be separated. On the provided photographs none completes a single one, and the ordering is unambiguous. The harder test was the informative one.

5. **Sample size decided a headline claim, and got it wrong.** The

low-texture sweep at four puzzles per level said learning buys robustness; at twenty it said the opposite (§6.3).

6. Representation beat capacity. The Siamese wins on accuracy through a

richer input - the full colour strip versus eight pooled bins - not through its extra parameters; the graph model reaches a competitive AUC on 3.5x fewer parameters and 50x less training time.

7. Compare against the baseline your own project runs. The classical

baseline was configured wrongly twice here, each time in the direction that flattered the learned methods, and each time because Milestone 1 had improved and this milestone had not followed.

8. The accuracy gained is specific to this jigsaw. Training on photographs

of one puzzle buys accuracy on that puzzle and would not transfer to another, where the classical measure still would. The gain is real and it is what the brief asks for; the limitation should be stated alongside it.

7. Reproducing

```
pip install -r requirements.txt # includes torch and torch-geometric
python main_milestone2.py --real --scaling --hard --figures # the headline run
python main_milestone2.py --scaling # generated puzzles, for the size study
python tests/run_tests.py ml_models # the Milestone 2 tests
```

--real trains on the **provided photographs** under detection/, as Task 1 requires, and is what results/milestone2/ holds. Without it the run uses generated puzzles and writes to results/milestone2_generated/ instead, so it cannot overwrite the committed results.

results/milestone2/ holds the committed run: dataset.json (split and class balance), training.json (architectures, hyper-parameters, per-epoch curves), comparison.json (the three-way comparison), scaling.json (accuracy on the provided photographs, all 5x7) and scaling_generated.json (the puzzle-size study of §6.2), texture_sweep.json and real_photographs.json (the two hard conditions), predicted_matches.json (the four required per-pair outputs) and the two trained checkpoints.

7.1 The reconstructed images

--figures writes results/milestone2/figures/: three charts summarising the tables above - training_curves.png, test_split_comparison.png, texture_sweep.png - and eight reconstructed images, real_0718-10... to real_0718-19..., one per dataset photograph. Each shows the three methods side by side on the same photograph, captioned with its measured neighbour and position accuracy; figures/index.json records the same numbers.

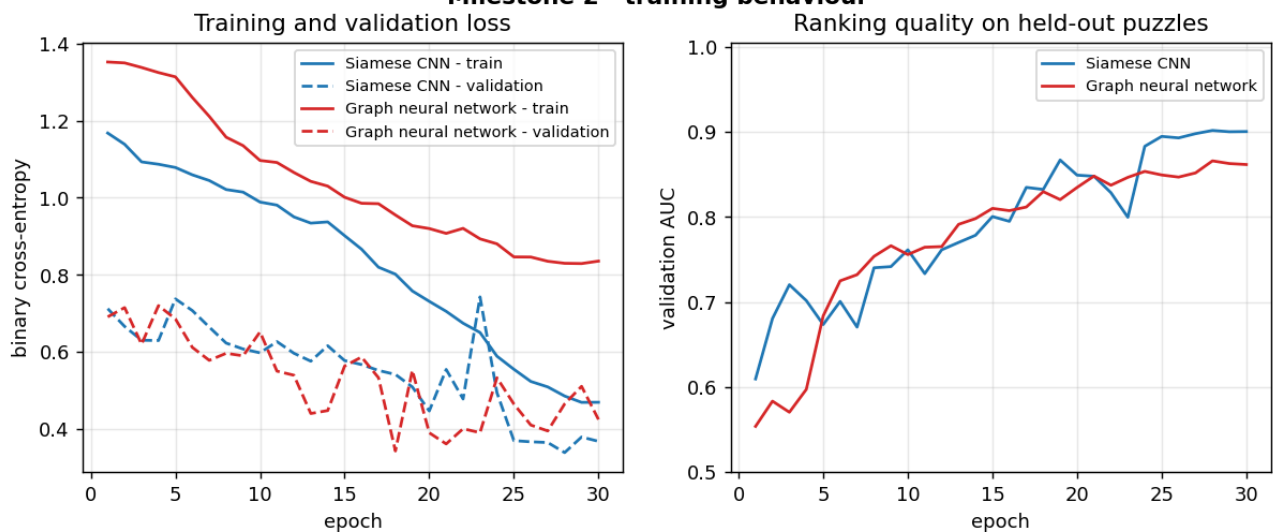
Only the real photographs are drawn. The generated puzzles are what the models are trained and scored on, and §6.1-6.3 report those numbers in full, but a picture of a generated puzzle being solved shows nothing its numbers do not - so the images kept here are of the actual jigsaw.

That makes every reconstructed image in this report a failure, which is the point of including them. The captions carry the numbers deliberately: the panels show thirty-five pieces tiled into a clean rectangle with barely a fifth of them in the right cell, and that is exactly the arrangement a viewer would otherwise take for a success.

Appendix - Figures

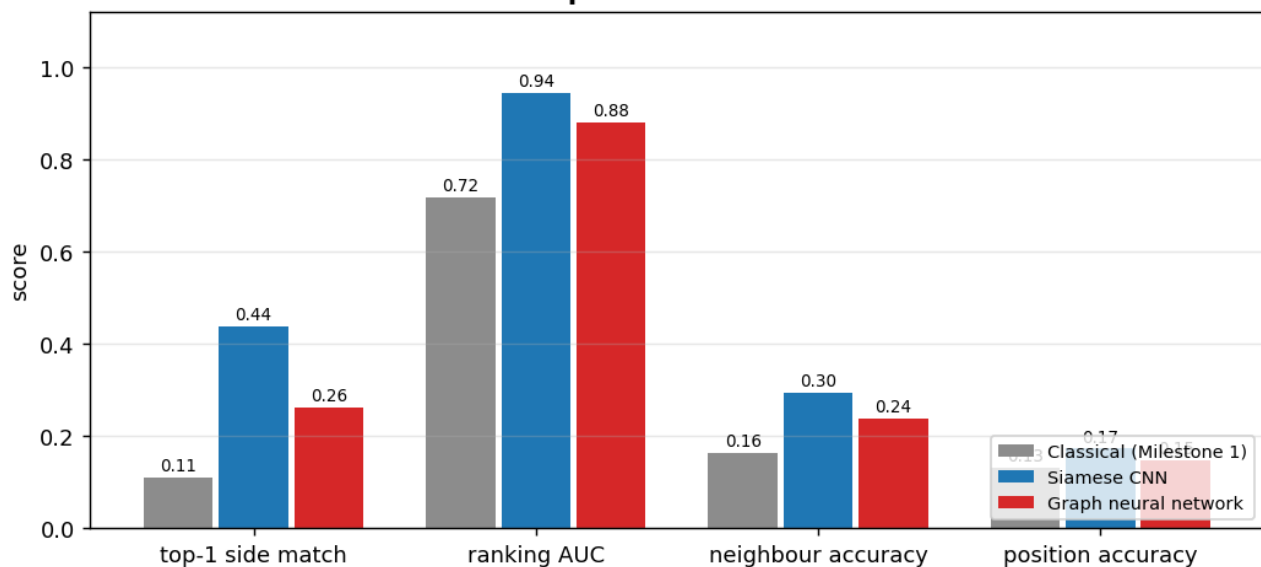
Every figure below is produced by `python main_milestone2.py --hard --figures` and written to `results/`.

Milestone 2 - training behaviour



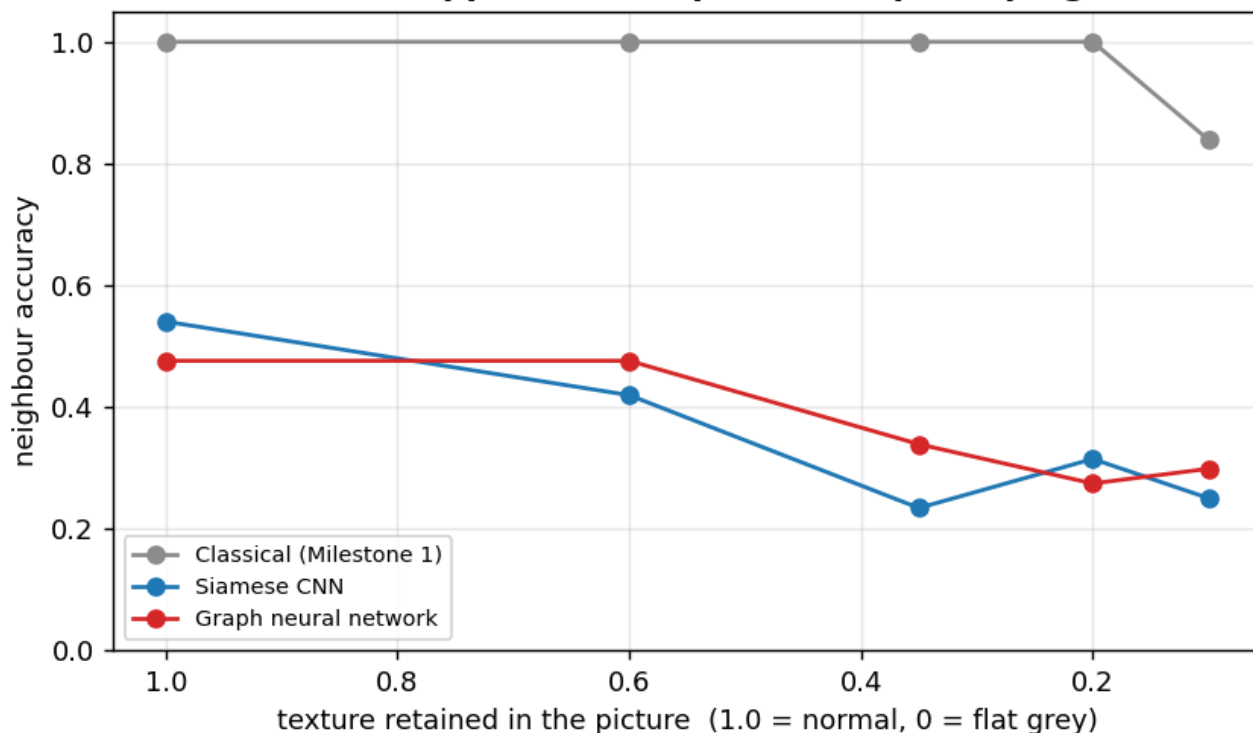
`milestone2/figures/training_curves.png` — Task 3 - training behaviour: cross-entropy per epoch and the validation AUC both models were selected on. The train and validation curves stay together, which is why both are diagnosed healthy.

Unseen test puzzles - all three methods



`milestone2/figures/test_split_comparison.png` — Task 5 - the unseen test split. Every method reconstructs every puzzle, so this comparison is saturated and cannot rank them; only the ranking quality (AUC) separates the models here.

What happens as the picture stops helping



milestone2/figures/texture_sweep.png — Task 5 - neighbour accuracy as the picture's texture is faded towards flat grey. The hand-written measure collapses at texture 0.10; the Siamese network does not.

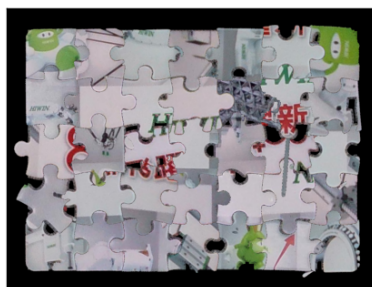
Dataset photograph - 0718-10_Color_png.rf.f269500b6bdb1b871bfc5b9c53814026.jpg

Classical (Milestone 1)



neighbour 0.17 position 0.09

Siamese CNN



neighbour 0.24 position 0.23

Graph neural network



neighbour 0.21 position 0.14

milestone2/figures/real_0718-10_Color_png.rf.f269500b6bdb1b871bf.png — Task 5 - a dataset photograph, reconstructed by all three methods. None of them solves it: the pieces tile into a neat rectangle and hardly any of them are in the right cell.

Dataset photograph - 0718-14_Color_png.rf.7ba095874a51a7e4c9516427b281ed77.jpg

Classical (Milestone 1)



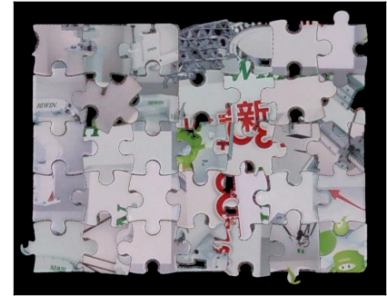
neighbour 0.19 position 0.06

Siamese CNN



neighbour 0.33 position 0.31

Graph neural network



neighbour 0.28 position 0.14

milestone2/figures/real_0718-14_Color_png.rf.7ba095874a51a7e4c95.png — The photograph on which the Siamese network does best (neighbour 0.28 against the classical measure's 0.12) - still far from a solution.

Dataset photograph - 0718-18_Color_png.rf.7ce49fe96d1498ecbab4d952cca0c910.jpg

Classical (Milestone 1)



neighbour 0.29 position 0.06

Siamese CNN



neighbour 0.41 position 0.46

Graph neural network



neighbour 0.19 position 0.06

milestone2/figures/real_0718-18_Color_png.rf.7ce49fe96d1498ecbab.png — A third photograph. Across all eight the learned models report much better seam quality while placing no more pieces correctly, which is what being confidently wrong outside the training distribution looks like.